



Софийски университет „Св. Кл. Охридски”

Факултет по математика и информатика

Курсов Проект

на тема: „Модел за откриване на тумор в мозъка”

Студент: **Георги Николаев Атанасов Ф.Н. ЗМИ0800092**

Курс: 4-ти, Учебна година: 2024/25

Преподаватели: **проф. Иван Койчев:**

=====

Декларация за липса плагиатство:

- Плагиатство е да използваш, идеи, мнение или работа на друг, като претендираш, че са твои. Това е форма на преписване.
- Тази курсова работа е моя, като всички изречения, илюстрации и програми от други хора са изрично цитирани.
- Тази курсова работа или нейна версия не са представени в друг университет или друга учебна институция.
- Разбирам, че ако се установи плагиатство в работата ми ще получа оценка “Слаб”.

30.1.25 г.

Подпис на студента:

Съдържание

1	УВОД	2
2	ПРЕГЛЕД НА ОБЛАСТТА	2
3	ПРОЕКТИРАНЕ	3
4	РЕАЛИЗАЦИЯ, ТЕСТВАНЕ/ЕКСПЕРИМЕНТИ	4
4.1	ИЗПОЛЗВАНИ ТЕХНОЛОГИИ, ПЛАТФОРМИ И БИБЛИОТЕКИ.....	4
4.2	РЕАЛИЗАЦИЯ/ПРОВЕЖДАНЕ НА ЕКСПЕРИМЕНТИ.....	6
5	ЗАКЛЮЧЕНИЕ	7
6	ИЗПОЛЗВАНА ЛИТЕРАТУРА.....	8

1 Увод

Целта на проекта е да се реализира и обучи модел, който да може по подадена снимка от ядрено-магнитен резонанс на мозък, да определи дали пациентът има тумор и ако да – какъв вид. Съответно, видовете които моделът трябва да може да разпознава са:

- Глиома (на англ. *glioma*) – тумор, който засяга глиалната тъкан на мозъка и засягащ нервната система. Може да е както доброкачествен, така и злокачествен
- Менингиом (на англ. *meningioma*) – тумор, който се развива в менингите (обвивки на главния и гръбначен мозък). В повечето случаи е доброкачествен, много рядко е злокачествен
- Аденом на хипофизата (на англ. *pituitary*) – туморно образование, произлизащо от клетките на хипофизната жлеза. Среща се при около 20% от хората и в 99,9% от случаите е доброкачествено

Основната задача на проекта е да се реализира модел, без използването на външни библиотеки за реализацията на архитектурата, който да има точност от поне 65%. Моделът трябва да бъде достатъчно достъпен, така че да може да се обучи и използва на машини, които могат да нямат специализиран хардуер за обучение на модели, т.е. на средно-статистически компютър.

2 Преглед на областта

Проектът се свежда до това, да бъдат анализирани изображения и да може да се извлече необходимата информация от тях, така че да може да се направи заключение за диагнозата на пациента. Има много различни подходи, по които може да се постигне това, като някои от тях са:

1. Невронни мрежи – стандартните невронни мрежи или т.нар. многослойни персептрони са един от по-наивните модели за разпознаване на характеристики в изображения. Понеже те обработват изображенията, както биха обработили линейни данни, много често при тях се наблюдават трудности при обучението, за което са необходими много данни и ресурси, и често отделните характеристики от изображенията не биват научени добре.
2. Конволюционни невронни мрежи – това са невронни мрежи, които са специализирани в анализа на изображения. Те включват конволюционни слоеве, които учат конкретни характеристики и обединяващи слоеве, които намаляват размерността и шума от изображенията, което помага на мрежата да не се пренагажда към тренировъчните данни.
3. Алгоритми за компютърно зрение, като например *SIFT (Scale-Invariant Feature Transform)* – това е алгоритъм, който може да открива и сравнява

локални характеристики от изображения. Тези характеристики се базят в някаква база от данни и при получаване на заявка за анализ на изображение, неговите характеристики се сравняват с тези в базата от данни и по този начин може да се класифицира на база на най-подобното изображение.

4. *eXtreme Gradient Boosting (XGBoost)* – това е алгоритъм за ансамблов учене, който може да се модифицира, така че да използва част от гореспоменатите алгоритми, които да се научат да разпознават само част от характеристиките на изображенията, и да се комбинират в един общ модел.

За реализацията на проекта е използвана конволюционна невронна мрежа, понеже това е един утвърден модел в анализа на изображения, който от една страна е по-точен от обикновените невронни мрежи, но и също така не изисква толкова много системни ресурси, колкото останалите алгоритми.

3 Проектиране

Данните, върху които моделът се обучава и тества са взети от множеството от данни *Brain tumor dataset* от *Kaggle (Backert)*. За архитектурата на мрежата са реализирани следните слоеве:

- Конволюционен слой – отговаря за извличането на характеристики от изображенията. Има само 1 канал, понеже ЯМР снимките са черно-бели. Има възможност за добавяне на произволен брой филтри (или ядра), които представляват матрици, които се „плъзгат“ през изображенията и научават различните характеристики от тях. Резултатът от конволюционния слой са карти на характеристиките, като броят на картите е равен на броя на филтрите.
- Обединяващ слой – стои след конволюционния слой и служи за намаляване на размерността на картите на характеристиките. Това става, като всяка карта се разделя на региони, представляващи подматрици, и се извличат само част от данните в тези региони. За целите на проекта е реализирано максимално обединяване, т.е. от всеки регион се извличат най-големите стойности. Този метод се характеризира с допълнителното свойство, че премахва шума от изображенията. Резултатът от обединяващия слой са матрици с извлечените стойности, които са толкова на брой, колкото са и входните карти на характеристиките.
- Изравняващ (сплескващ) слой – стои след обединяващ и преди напълно свързан слой и служи за трансформация на изхода от обединяващия към формата на входа на напълно свързания слой. Понеже изхода от обединяващия слой е списък от матрици, изравняващия слой трябва да

„сплеска“ тези матрици до единичен вектор, който може да служи за вход на напълно свързания слой.

- Изхвърлящ слой – стои между два напълно свързани слоя и има задачата да изхвърли част от резултата на първия слой с цел да предотврати пренагаждането към тренировъчните данни.
- Напълно свързан слой – представлява характерния за персептрона слой, който има връзки от всеки вход към всеки изход. Чрез него могат да се учат комбинации от извлечените по-рано характеристики по същия начин, както се обучават слоевете в стандартните невронни мрежи.

Чрез гореизброените слоеве конволюционната мрежа може ефективно да научава отделните характеристики от изображенията и техните комбинации, като в същото време не се пренагажда към тренировъчните данни.

Като допълнение, по време на обучение, мрежата пресмята загубата на ентропия за всяка епоха, чрез която може да се прави анализ до колко тя наистина се обучава.

След обучаването на модела се прави проверка за неговата точност, като се използват тестовите данни. Съотношението между тренировъчните и тестовите данни е 80:20. Също така се показва графика на промяната на загубата на ентропия и се визуализират няколко примерни изображения с техния реален и предположен от мрежата клас (диагноза).

4 Реализация, тестване/експерименти

4.1 Използвани технологии, платформи и библиотеки

Проектът е реализиран на езика *Python* като са използвани следните библиотеки:

- *Numpy* 2.2.0
- *Tensorflow* 2.18.0
- *Keras* 3.8.0
- *Matplotlib* 3.10.0

Tensorflow и *Keras* се използват единствено за зареждане на изображенията, тяхното стратифицирано разделяне на тренировъчно и тестово множество, както и за тяхното трансформиране – нормализиране на стойностите и преоразмеряване. *Matplotlib* се използва за визуализиране на графиката на промяна на загубата на ентропия и на примерните изображения с тяхното класифициране.

Моделът е обучаван на машина със следните характеристики:

- Процесор *AMD Ryzen CPU 5 5500u* с честота *2.10 GHz* и *turbo speed 4.0 GHz*
- *RAM* памет *16 GB*
- *L1 D-cache* и *L1 I-cache* по *32 KB* всеки, *L2 cache 512 KB* и *L3 cache 8 MB*

За обучението на модела са използвани 1024 тренировъчни и 256 тестови изображения, като това е около 1/5 от общия брой изображения в множеството от данни. Всяко изображение има реален размер от 128x128 пиксела, но моделът е обучен върху преоразмерени изображения с размер 32x32 пиксела. Причината да не се ползват всички данни и да се разглеждат умалени варианти на изображенията е времето, което е необходимо на модела да се обучи.

Самата мрежа включва:

1. Един конволюционен слой с 4 филтъра с размери 3x3. За активационна функция е използвана $ReLU(x) = \max\{0, x\}$, а за инициализиране на теглата на филтрите е използвана нормална Хи инициализация $\sim \mathcal{N}(0, n)$, където n е размерът на филтрите.
2. Един обединяващ слой с размер на регионите 2x2 и отстояние между тях от 2 клетки. При обратното разпространение на грешката, ако в един регион има няколко клетки със стойност равна на максималната стойност в региона, грешката се разпространява до всяка една от тях.
3. Един изравняващ слой.
4. Един напълно свързан слой с 900 неврона и 4 изхода, отговарящи на 4-те класа. За активационна функция е използвана $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$, а за инициализиране на теглата е използвана инициализация на Хавиер $\sim \mathcal{N}(0, 1/n)$, където n е броят на входните неврони.

Обучението на модела се случва за 50 епохи с темпо на обучение 0.2, което обаче намалява със 75% на всеки 7 епохи.

Като допълнение, с цел намаляване на пренагаждането на модела към тренировъчните данни, при обратното разпространение на грешката в напълно свързания и конволюционния слой е използвана $L2$ регуларизация, имаща формулата от Фигура 1.

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

L2 Regularization

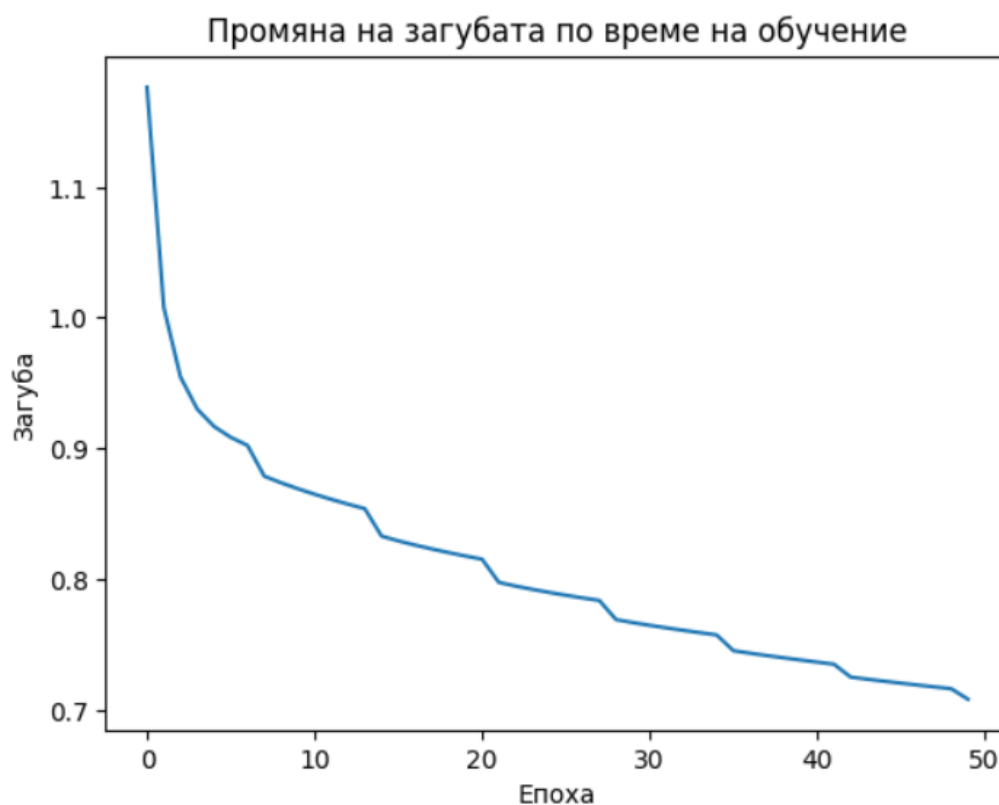
Фигура 1 Формула на $L2$ регуларизацията

4.2 Реализация/Провеждане на експерименти

По време на имплементацията са провеждани експерименти с по 32 тренировъчни изображения с размери 16x16 и 20 епохи. При тях, по-голяма точност даваше по-високо темпо на обучение от 0.7. В този случай обаче данните бяха недостатъчни, за да се обучи моделът, като точността варираше между 20% и 42%.

След това бяха направени опити със съответно 64, 128, 256 и 1024 изображения, като след 256 разликата в точността на модела беше сравнително ниска. Промяната на размера на изображенията от 16x16 на 32x32 добави около 10% към средната точност на модела, понеже той успяваше да научи по-добре характеристиките.

Бяха направени опити за промяна на архитектурата, като се добавят 2 и в следствие 3 напълно свързани слоя, но това не доведе до по-добри резултати, даже напротив – започна да се наблюдава осцилация на загубата на ентропия, която допълнително беше станала около 5 пъти по-висока.

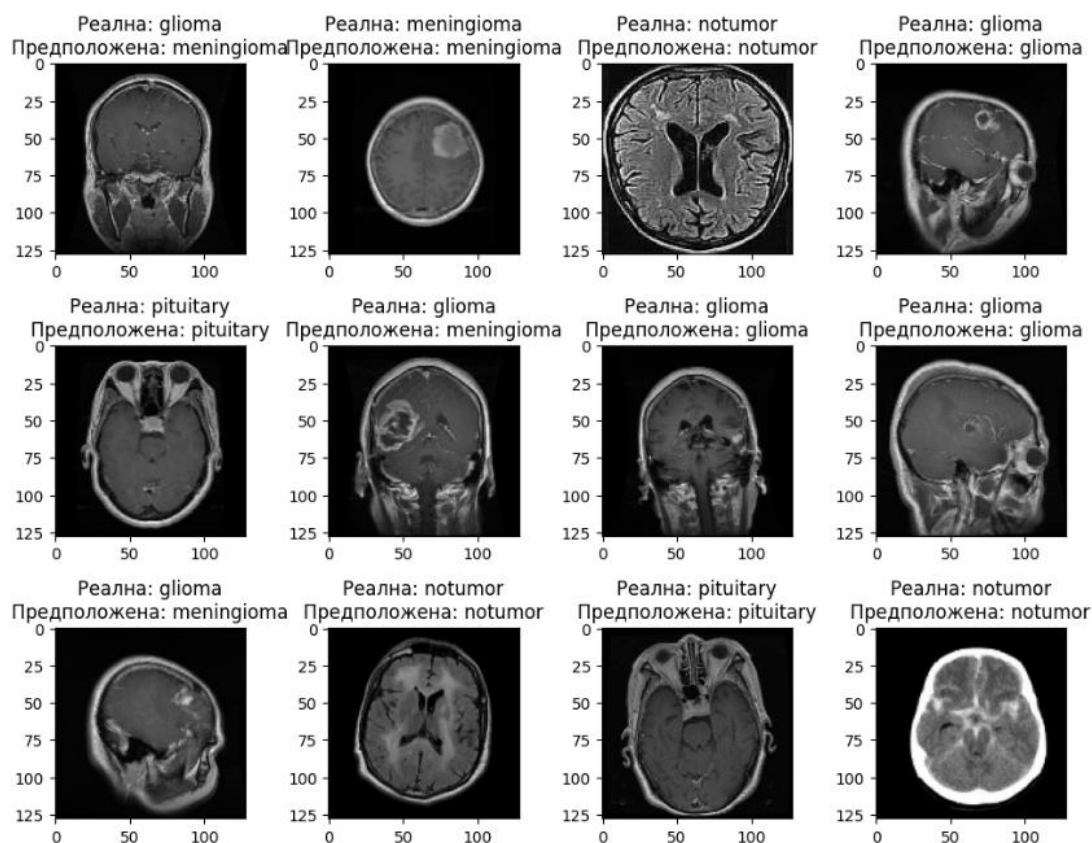


Фигура 2 Примерна графика на загубата на ентропия по време на обучение на финалния модел

С гореописаните хиперпараметри, моделът има точност между 71% и 77%, като това са приемливи резултати, като се има предвид ограничените ресурси на

машината, върху която се обучава модела (за справка, едно обучение на модела отнема около 30 минути).

Същата архитектура на модела беше реализирана и чрез библиотеките *Tensorflow* и *Keras*, които са доста по-оптимизирани и успяват да се обучат върху пълното множество от данни. Направените експерименти върху тази имплементация върху същите хиперпараметри, с разликата, че изображенията имаха размер 64x64, доведоха до точност от около 90%.



Фигура 3 Примерни резултати при употребата на обученния модел

5 Заключение

Постигнатите резултати от точност между 71% и 77% при гореописаните хиперпараметри потвърждават, че моделът успява да се обучава от използваните данни и да прави сравнително коректни предположения. За сравнение, ако моделът правеше произволни предположения, точността му трябваше да е 25%. Направените допълнителни експерименти с използването на библиотеките *Tensorflow* и *Keras* потвърждават възможността на модела да продължава да се обучава с по-голяма точност при по-добро използване на системните ресурси.

Като бъдещо развитие на проекта могат да се разгледат следните основни точки:

- Подобряване на употребата на системните ресурси, което да позволи обучението на модела върху цялото множество от данни и върху по-голям мащаб от изображенията
- Промяна на архитектурата на модела, която да включва повече конволюционни и напълно свързани слоя, с възможност за добавяна на изхвърлящи слоеве
- Използване на допълнителни методи за справяне с пренагаждането на модела към тренировъчните данни като например аугментация на изображенията

6 Използвана литература

Антон Христов, Мария Нишева, Димо Димов. (2018) „Въведение в конволюционните невронни мрежи.“ <https://sai-bg.com/wp-content/uploads/2021/08/5.-Paper-5-A.Hristov-M.Nisheva-D.Dimov-pages-27-38-AI-1-2018.pdf> последно посетен на 29.01.2025

Backert, T. (2024). *Brain tumor dataset*. <https://www.kaggle.com/datasets/tombackert/brain-tumor-mri-data> последно посетен на 29.01.2025

Saha, Sumit. (2018) "A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way." <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53> последно посетен на 29.01.2025

Solai, Pavithra. (2018) „Convolutions and Backpropagations.“ <https://pavisj.medium.com/convolutions-and-backpropagations-46026a8f5d2c> последно посетен на 29.01.2025